AALBORG UNIVERSITY

STUDENT REPORT

Computer Science (IT)
Aalborg University
http://www.cs.aau.dk

Title:
Trajectory Simplification for Large-Scale Movement Data

Theme:
Reliable Innovative System

Project Period:
Spring Semester 2026

Project Group:
cs-26-it-8-06@student.aau.dk

Participant(s):
Amardip Regmi
Ashok Kumar Sunuwar
Mandip Parajuli
Rajendra Niroula
Sanjay Raut
Saurav Tandukar

Supervisor(s):
Yumeng Song

Copies: 1

Page Numbers: 24

Date of Completion:
May 26, 2026

Abstract:

Trajectory data records how objects move over time and is collected by GPS devices, phones, vehicles, sports trackers, and other sensors. Because sensors sample frequently, trips can contain thousands of points and datasets can reach millions, creating storage, transmission, visualization, and analysis challenges. Trajectory simplification reduces point counts while preserving key movement information. This project focuses on large-scale simplification with an emphasis on reliability—stable, predictable behavior across diverse trajectories—so simplified traces retain turns, stops, and speed changes. Using the Microsoft GeoLife dataset, we evaluate classical and learning-based methods, with special attention to reinforcement-learning approaches that learn which points are important. The report covers problem statement, methodology, system design, implementation plan, expected findings, analysis strategy, and reliability considerations. Our main contribution is a critical-point-aware simplification approach that blends geometric error with movement behavior and presents a reliability-focused system view including preprocessing, fair baselines, multiple metrics, runtime analysis, and transparent limitations.

Contents

1 Introduction

Trajectory data is a list of locations where each location gives latitude and longitude and the exact time and date it was recorded. In recent time, GPS devices, smartphones, and sensors collect large amounts of movement data. These datasets can contain thousands or millions of points. This creates problems for storage, processing, transmission, and visualisation.

Trajectory simplification is one common technique used to handle this problem. The main idea is to reduce the number of trajectory points while preserving the important movement pattern of the original path. For example, points on a long straight road may be less important, while points around turns, stops, and speed changes may be more important.

The aim is to study how GPS trajectories can be reduced in size while still keeping the important shape and movement behaviour of the original trajectory.

1.1 Background

Trajectory data is important because it helps us understand how people, vehicles, or objects move. It can support many real-life systems, such as traffic monitoring, delivery planning, smart city development, and route analysis. However, large trajectory datasets are difficult to manage. If every GPS point is stored and processed, the system may become slow and expensive. So for solving this problem there is need to reduce unnecessary points. Trajectory simplification helps by keeping important points and removing redundant points.

Classical simplification methods mainly focus on geometric shape. Newer research also shows that time, speed, direction, and movement behaviour should be considered because shape alone may not fully represent the original movement.

1.2 Problem Statement

The main problem is that large-scale trajectory datasets contain too many points. This increases storage cost, slows down processing, and makes visualisation and analysis harder but also if we remove too many points it can damage the quality of the trajectory. For example, if we remove important points such as turns, stops, and speed changes the important behaviour of trajectory may get lost. This means the simplified trajectory may no longer represent the original movement correctly. Therefore, the key problem of this project is: How can we reduce the number of points in large GPS trajectories while preserving important movement information and keeping the simplified trajectory reliable?

1.3 Aims and Objectives

The main aim of this project is to study and develop a reliable trajectory simplification approach for large-scale movement data.

The objectives are:

- To understand the trajectory simplification problem and study both classical and learning-based methods.
- To compare and evaluate those methods on the GeoLife GPS dataset using appropriate error metrics.
- To preserve important movement features (turns, stops, speed changes) and assess each method's reliability, efficiency, and limitations.

1.4 Research Questions

This project is motivated by the following research questions :

- How can large GPS trajectories be simplified while keeping important movement information?
- Which simplification methods are suitable for reducing trajectory size?
- How can we measure the quality of a simplified trajectory?

1.5 Scope and Limitations

This project focuses on GPS trajectory simplification using large-scale movement data. The main dataset considered is the Microsoft GeoLife GPS trajectory dataset. The project mainly studies batch simplification, where the full trajectory is available before simplification.

The project does not aim to solve all trajectory-related problems. It focuses mainly on reducing points and evaluating quality. Full real-time implementation and advanced reinforcement learning training is not in our main scope. However, reinforcement learning is discussed as an important research direction.

2 Literature Review

2.1 Overview of Trajectory Simplification

Trajectory simplification means reducing the number of points in a trajectory while keeping its important characteristics. The simplified trajectory should be smaller, but it should still look and behave similarly to the original trajectory.

Wang, Long, and Cong[6] explain that trajectory data is often collected continuously by sensors and stored on servers, which creates a large data volume. They state that simplification is useful because not all points contain equal information. For example, if movement happens in a straight line at constant speed, many middle points may be unnecessary.

This idea is very important for this project because the project also assumes that some points are not necessary, while others are critical for preserving the meaning of movement.

2.2 Classical Simplification Algorithms

Classical trajectory simplification algorithms usually use geometric rules to decide which points should be kept or removed. These methods are normally easier to understand and implement.

One well-known method is the Douglas-Peucker algorithm. It keeps points that have a large perpendicular distance from a simplified line segment. This means it is useful for preserving the geometric shape of a trajectory but it mainly focuses on shape and does not directly consider time, speed, or stops [1].

Another method is the Sliding Window algorithm. It processes points in sequence and keeps points when the approximation error becomes too large. This method is simple and efficient, but it may miss globally important points because it mainly uses local information.

Other methods such as Visvalingam-Whyatt, Reumann-Witkam, and SQUISH also simplify trajectories using different rules. SQUISH is especially useful for GPS compression because it removes low-priority points and updates neighboring priorities. However, it is still mostly based on predefined rules rather than learning from data [5, 4].

Zhang et al.[7] studied trajectory simplification experimentally and showed that no single method is best for all situations. Different algorithms can perform differently depending on the quality metric used. This supports the need to compare multiple methods instead of relying on only one algorithm .

2.3 Reinforcement Learning Based Algorithms

There has been a lot of research on the exploration of learning-based methods for trajectory simplification. Reinforcement learning enables a system to learn from the data which trajectory points are important, rather than relying solely on fixed human-designed rules.

Wang, Long, and Cong [6] proposed a reinforcement learning-based method called RLTS. In this method, trajectory simplification is treated as a decision-making problem. The system

observes the current state of the trajectory and decides which point should be removed. The decision is rewarded on the basis of the quality of the simplified trajectory.

They also proposed RLTS-Skip, which can skip points that carry little information. This improves efficiency because the system does not need to spend time carefully processing every unimportant point. Their work shows that reinforcement learning can reduce simplification error while still maintaining reasonable efficiency.

This is relevant to the project because it shows that learning-based methods can be more adaptive than classical methods. However, reinforcement learning is also more complex because it needs training data, parameters, and careful evaluation.

2.4 Advantages and Limitations of Existing Techniques

Existing trajectory simplification methods each have clear strengths and weaknesses.

Classical methods are simple, fast, and easy to understand. For example: Douglas–Peucker preserves route shape well, while Sliding Window works better for ordered data. However, both focus mainly on geometric error and can remove movement-important points such as stops, turns, or speed changes.

Learning-based methods can learn from data and handle complex movement patterns better, but they are harder to build and need more training, tuning, and computing power. This project uses simple classical methods as baselines and adds a critical-point-aware step to keep important movement points like turns, stops, and speed changes. That makes the work practical, easy for to follow, and more reliable for real-world use.

3 Methodology

3.1 Overall System Workflow

The system follows a structured workflow for trajectory simplification and evaluation. Raw GPS trajectories are first loaded from the GeoLife dataset and then preprocessed to remove duplicate records, filter unrealistic movement speeds, and reduce spatial outliers. During preprocessing, additional trajectory properties such as speed, stops, and direction changes are also computed. The cleaned trajectories are then simplified using both the baseline algorithms and the proposed semantic-aware method under a common target number of retained points. The simplified outputs are evaluated using compression, geometric, and time-aware metrics, and the final results are compared using tables, plots, and interactive visualisations. Figure 1 shows the overall workflow of the proposed system.

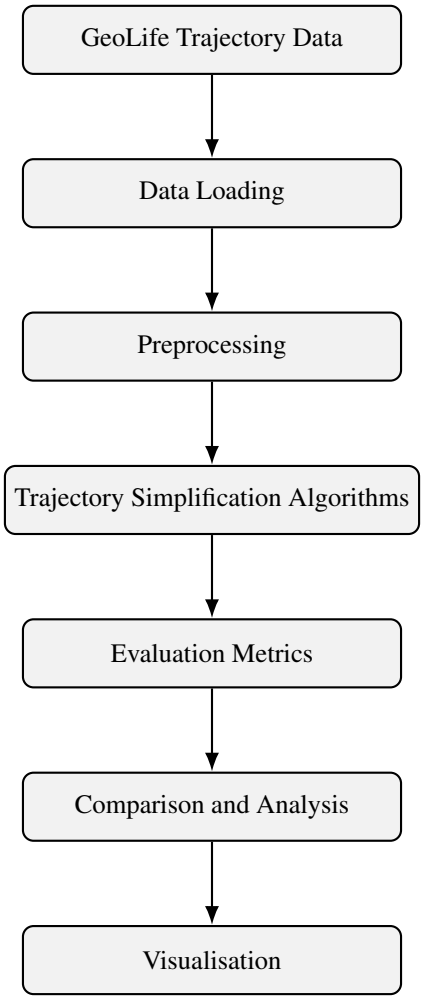

Figure 1: Overall workflow of the trajectory simplification system.

3.2 Overview of the GeoLife Dataset

This project uses the Microsoft GeoLife GPS trajectory dataset as the main source of trajectory data. According to the GeoLife user guide, the dataset was collected from 182 users over more than five years, from April 2007 to August 2012, and contains 17,621 trajectories with a total travel distance of 1,292,951 km and a total duration of 50,176 hours. The dataset contains diverse outdoor movements, including commuting, shopping, sightseeing, dining, hiking, and cycling, and the majority of the data was recorded in Beijing, China. In addition, 91.5% of the trajectories are logged in dense form, typically every 1–5 seconds or every 5–10 metres, which makes the dataset particularly suitable for trajectory simplification research [3].

Each trajectory is stored as a `.plt` file in a user-specific folder. The GeoLife user guide states that the first six lines of each file are metadata and can be ignored. Each subsequent line represents one GPS point and contains latitude, longitude, altitude, and timestamp information. The timestamp is provided both as a fractional day value and as separate date and time strings, and the guide notes that all timestamps are stored in GMT in Version 1.3 of the dataset [3].

3.3 Preprocessing Pipeline

Before simplification, each trajectory is cleaned through a sequence of filtering steps applied uniformly to all trajectories. This ensures that all algorithms operate on the same cleaned input data.

Duplicate removal. Rows with identical timestamps within the same trajectory file are removed. Duplicate timestamps can create zero-duration intervals, which may affect speed computation and later trajectory analysis.

Speed-based filtering. Consecutive-point speeds are computed using the Haversine distance between neighbouring points divided by the elapsed time. Points implying unrealistic movement speeds are removed using a configured maximum-speed threshold, since such values are likely to represent GPS errors or non-ground movement.

Spatial outlier removal. Step lengths between consecutive points are analysed to detect unusually large spatial jumps. A robust Median Absolute Deviation (MAD)-based threshold is used to reduce the influence of isolated GPS noise before simplification.

Minimum-length filtering. A minimum-length filter is applied so that very short trajectories are excluded from the experiments. This ensures that the selected trajectories contain enough points for meaningful comparison across the tested compression ratios.

Temporal sorting. All remaining points are sorted by timestamp to preserve the correct chronological order of movement.

After preprocessing, the cleaned trajectories are used as the common input for all simplification methods, which helps ensure a fair comparison across algorithms.

3.4 Baseline Algorithms

The project compares the proposed method against four baseline algorithms that represent different simplification strategies: recursive geometric simplification, area-based simplification, strip-based simplification, and a motion-aware greedy policy. A large comparative study shows that no single simplification algorithm dominates across all quality metrics, so including several baselines is methodologically justified [7].

3.4.1 Douglas–Peucker (DP)

Douglas–Peucker is the main geometric baseline used in this project. Given an ordered curve $C = (P_1, \ldots, P_n)$, the algorithm connects the first and last point and finds the interior point with the maximum perpendicular distance to the line segment joining them. This maximum deviation is defined as

$$d_{\max} = \max_{i=2,\ldots,n-1} d\left(P_i, \overline{P_1 P_n}\right) \tag{1}$$

where $d(P_i, \overline{P_1 P_n})$ denotes the perpendicular distance from point P_i to the segment $\overline{P_1 P_n}$. If $d_{\max} \leq \varepsilon$, all interior points are discarded; otherwise, the farthest point is retained and the algorithm is applied recursively to the two resulting sub-curves. This makes Douglas–Peucker a strong geometric shape-preserving method, but it does not explicitly preserve stops, turns, or temporal behaviour [1, 7].

3.4.2 Visvalingam–Whyatt (VW)

Visvalingam–Whyatt removes points in order of increasing effective triangle area. For each interior point P_i, the importance is defined as the area of the triangle formed by P_{i-1}, P_i, and P_{i+1}:

$$A_i = \frac{1}{2} \left| x_{i-1}(y_i - y_{i+1}) + x_i(y_{i+1} - y_{i-1}) + x_{i+1}(y_{i-1} - y_i) \right| \tag{2}$$

Points with the smallest A_i are removed first, producing a smooth simplification that often performs well visually. Like Douglas–Peucker, however, Visvalingam–Whyatt is geometry-focused and does not explicitly model temporal or semantic trajectory features [5, 7].

3.4.3 Reumann–Witkam (RW)

Reumann–Witkam is a strip-based trajectory simplification method. Starting from two consecutive anchor points, the algorithm defines a corridor of width ε and removes subsequent points while they remain inside this corridor:

$$d_{\perp}(v_i, \text{line}(v_s, v_{s+1})) \leq \varepsilon \tag{3}$$

where $\text{line}(v_s, v_{s+1})$ denotes the infinite line passing through v_s and v_{s+1}. When a point falls outside the corridor, a new segment is started. This makes the method fast and suitable for local simplification, but it may perform poorly when the trajectory contains strong global curvature or semantically important local events such as stops and turns [4, 7].

3.4.4 Greedy Policy (GP)

Greedy Policy is a deterministic baseline inspired by reinforcement-learning trajectory simplification. Instead of learning a policy, it assigns each interior point a score that combines geometric deviation and motion change, then retains the highest-scoring points under the fixed point limit. In the implementation, the point score is defined as

$$v(i) = \alpha \cdot geo\_dev(i) + (1-\alpha) \cdot motion\_change(i) \tag{4}$$

with $\alpha = 0.5$. This method introduces motion sensitivity without requiring a training phase, but unlike the proposed method it does not explicitly model stop duration or sampling irregularity [6, 7].

3.5 Proposed Method

The proposed method in this project is designed to preserve behaviourally important points rather than relying only on geometric deviation. In real GPS trajectories, some points are important because they represent meaningful motion events such as turns, stops, speed changes, or irregularly sampled regions, even if they do not have large perpendicular geometric error. To account for this, each point is assigned an importance score based on four semantic and motion-related components: turn significance, stop significance, speed-change significance, and sampling irregularity significance.

3.5.1 Overview

The proposed method operates in three main stages:

1. **Importance scoring:** compute a multi-dimensional importance score for each trajectory point,
2. **Point selection:** retain the highest-scoring points under a fixed point limit,
3. **Geometric refinement:** refine the selected set to avoid large local geometric errors.

This design allows the method to preserve semantically important trajectory behaviour while still maintaining acceptable geometric quality.

3.5.2 Importance Scoring

The importance of a point p_i is defined as:

$$I(p_i) = w_{\text{turn}} S_{\text{turn}}(p_i) + w_{\text{stop}} S_{\text{stop}}(p_i) + w_{\text{speed}} S_{\text{speed}}(p_i) + w_{\text{irr}} S_{\text{irr}}(p_i) \tag{5}$$

where the weights sum to 1.0. Each score has a simple role:

- **Turn score:** gives higher importance to points near strong direction changes.

- **Stop score:** gives higher importance to points inside stop regions, especially long stops.
- **Speed-change score:** highlights points where speed changes significantly.
- **Irregularity score:** preserves points recorded after unusually large sampling gaps.

3.5.3 Point Selection

Given a target number of retained points k, the method always keeps the first and last point of the trajectory. The remaining points are ranked according to their importance score, and the top-$(k-2)$ interior points are selected. This produces an initial simplified trajectory that prioritises semantically important locations.

3.5.4 Geometric Refinement

After the initial point selection, an optional geometric refinement step is applied. If the local geometric error between two selected points becomes too large, the intermediate point with the largest error may be added, provided that the target number of retained points allows it. This step helps prevent the simplified trajectory from becoming geometrically unrealistic while still preserving semantic features.

3.6 Evaluation Metrics

The simplified trajectories are evaluated using compression, geometric error, and time-aware error measures. A broad experimental study of trajectory simplification recommends using multiple metrics because different algorithms are biased toward different notions of quality [7].

3.6.1 Compression Ratio

The project reports the reduction factor as:

$$CR = \frac{|T|}{|T'|} \tag{6}$$

where $|T|$ is the number of original points and $|T'|$ is the number of simplified points. Larger values indicate stronger compression [7].

3.6.2 Perpendicular Euclidean Distance (PED)

For a point $p_m = (x_m, y_m, t_m)$ and its anchor segment $\overrightarrow{p_s p_e}$, the standard PED definition is:

$$PED(p_m) = \frac{|(y_e - y_s)x_m - (x_e - x_s)y_m + x_e y_s - y_e x_s|}{\sqrt{(y_e - y_s)^2 + (x_e - x_s)^2}} \tag{7}$$

PED measures geometric deviation from the simplified polyline and is the natural metric for Douglas–Peucker-style methods [7].

3.6.3 Synchronised Euclidean Distance (SED)

SED accounts for time by comparing an original point p_m with its synchronised point $p'_m = (x'_m, y'_m, t_m)$ on the simplified segment $\overrightarrow{p_s p_e}$. The synchronised point is obtained by linear interpolation:

$$x'_m = x_s + \frac{x_e - x_s}{t_e - t_s}(t_m - t_s), \qquad y'_m = y_s + \frac{y_e - y_s}{t_e - t_s}(t_m - t_s) \tag{8}$$

and the corresponding distance is:

$$SED(p_m) = \sqrt{(x_m - x'_m)^2 + (y_m - y'_m)^2} \tag{9}$$

SED is widely used in time-aware trajectory simplification because it reflects both spatial and temporal consistency [7, 2].

3.6.4 Hausdorff and Fréchet Distances

The project also uses Hausdorff distance and discrete Fréchet distance as global trajectory similarity measures. Hausdorff distance reflects the worst-case deviation between two point sets, while Fréchet distance respects the order of points along the trajectories and is often interpreted as the minimum "leash length" required to traverse both curves simultaneously. These measures are standard in trajectory similarity and simplification evaluation [7].

4 System Architecture and Implementation

4.1 Architecture Diagram

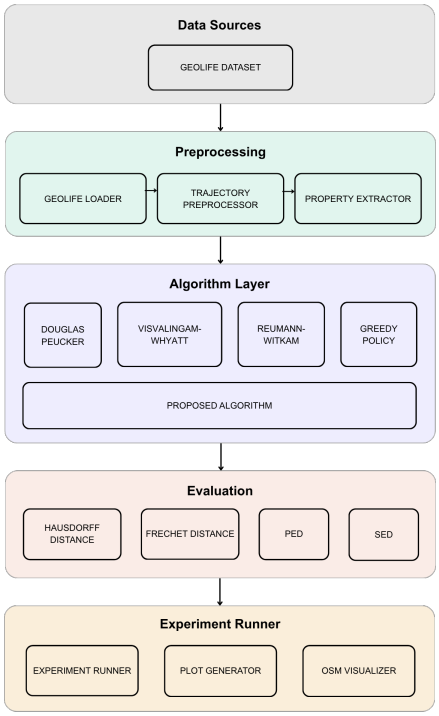

Figure 2: System Architecture

4.2 Implementation

4.2.1 Preprocessing — GeoLife Data

The preprocessing pipeline is implemented in `src/utils/preprocess_geolife.py` and `src/utils/geolife_loader.py`. It loads raw GPS trajectories from the GeoLife dataset, cleans them, and serialises the result for use by all downstream experiments.

4.2.1.1 Data loading. The `GeoLifeLoader` class reads `.plt` files from the standard GeoLife directory layout (`Data/`*XXX*`/Trajectory/`). Each file is parsed by skipping the first six header lines and reading seven columns: latitude, longitude, a constant zero field, altitude, a fractional day count, date, and time. The date and time columns are merged into a single `timestamp` field, yielding a DataFrame with columns `lat`, `lon`, `alt`, and `timestamp`.

4.2.1.2 Coordinate transformation. All distance calculations use the Haversine formula with Earth's mean radius $R = 6{,}371{,}000$ m. Point-to-segment perpendicular distances project the query point onto the segment using a dot-product parameterization in degree space (valid

for short segments), then apply Haversine to the projected point. The Fréchet distance computation uses a faster flat-Earth approximation—scaling degree differences by 111,320 m/deg and $111{,}320\cos(\bar{\phi})$ m/deg for longitude (where $\bar{\phi}$ is the mean latitude)—accurate to within 0.3 % for trajectories smaller than ≈ 300 km.

4.2.1.3 Outlier removal. Two filtering passes are applied. First, individual GPS points whose instantaneous speed to the next point exceeds 80 m/s (≈ 288 km/h, `MAX_VALID_SPEED_MS`) are discarded as noise. Second, entire trajectories are excluded if their mean speed exceeds this threshold, or if they overlap with a user-provided *airplane* transport-mode label (checked via a standard interval-overlap test on `timestamp` ranges). After cleaning, trajectories with fewer than 50 points are discarded. The processed trajectories are saved as a Python pickle (`data/processed/trajectories.pkl`) alongside per-trajectory property CSVs.

4.2.2 Simplification Algorithms

All algorithms are implemented in `src/algorithms/` and exposed through a unified `simplify_with_budget()` dispatcher accepting a trajectory, an algorithm name, and a target point count. Algorithms that accept a distance tolerance ε rather than a direct point budget are wrapped with a binary search (20 iterations, $\varepsilon \in [0, 1000]$ m) that finds the smallest ε producing at most the target number of points; an exact-padding step then re-inserts the highest-error excluded points to reach the count precisely.

4.2.2.1 Douglas-Peucker (DP). The classic recursive divide-and-conquer algorithm. Starting from the chord between the first and last points, the interior point with maximum perpendicular distance is identified. If this distance exceeds ε, the trajectory is split at that point and the procedure is applied recursively to each half; otherwise only the endpoints are retained. Complexity is $O(n^2)$ worst case, $O(n \log n)$ average. The algorithm is purely geometric: it ignores timestamps, speeds, and semantic features.

4.2.2.2 Visvalingam-Whyatt (VW). Area-based simplification. Each interior point is assigned an effective triangle area computed from its two neighbours. Points are removed in order of increasing area until the target count is reached.

4.2.2.3 Reumann-Witkam (RW). Corridor-based method. Starting from the first point, a strip of width ε is defined around the segment to the next point. All subsequent points within the strip extend the corridor; the first point outside triggers a new anchor at the last inside point, and the process repeats. Complexity is $O(n)$.

4.2.2.4 Greedy Policy (Greedy First). An RL-inspired deterministic baseline that assigns each interior point a scalar value

$$v(i) \;=\; \alpha \cdot g(i) \;+\; (1-\alpha) \cdot m(i), \tag{10}$$

where $g(i)$ is the perpendicular distance from p_i to the chord $p_{i-1} \to p_{i+1}$ (normalised across the trajectory), and $m(i)$ is the average of normalised bearing change and normalised speed change at p_i. The top $(k-2)$ interior points by value are retained together with the mandatory endpoints. Default $\alpha = 0.5$. Complexity is $O(n)$.

4.2.2.5 Proposed Algorithm. The proposed method combines five importance components with learned weights:

Table 1: Importance components of the proposed simplification algorithm.

Component	Weight	Description
Turn score	0.25	Direction change magnitude (bearing difference), smoothed over a 3-point window and boosted by local variance
Stop score	0.25	Duration of low-speed region (< 1 m/s for ≥ 30 s); longer stops score higher
Speed change	0.15	Absolute speed change, smoothed and normalised
Irregularity	0.15	Time gap relative to the median interval; sparse points score higher

The combined importance score is:

$$\text{importance}(p_i) = w_{\text{turn}}\,\tau(i) + w_{\text{stop}}\,s(i) + w_{\text{speed}}\,\delta(i) + w_{\text{irr}}\,\iota(i). \tag{11}$$

After initial selection of the top-k points by Eq. (11), an iterative geometric refinement pass inserts additional points into any gap whose worst-case perpendicular error exceeds an adaptive threshold (1 % of the trajectory's spatial diagonal, floored at 2 m), until the budget is exhausted or all gaps are below the threshold. If refinement overshoots the budget, the lowest-importance interior points are removed to restore the exact target count.

4.2.3 Error Metrics Implementation

All metrics are implemented in `src/metrics/evaluation_metrics.py` and called through `compute_all_metrics()`.

4.2.3.1 PED — Perpendicular Euclidean Distance. For each original point, the minimum perpendicular distance to any segment of the simplified trajectory is computed. PED is the mean of these distances across all original points.

4.2.3.2 SED — Synchronized Euclidean Distance. The simplified trajectory is interpolated to the original timestamps via linear interpolation in degree space. SED is the mean Haversine distance between each original point and its time-synchronised counterpart on the simplified trajectory.

4.2.3.3 Hausdorff Distance.

$$H(A,B) = \max\big(h(A,B), h(B,A)\big), \quad h(A,B) = \max_{a\in A}\min_{b\in B} d(a,b). \tag{12}$$

Each pairwise distance d uses the Haversine formula.

4.2.3.4 Fréchet Distance. Computed with dynamic programming over an $N \times M$ coupling table:

$$F(i,j) = \max\big(d(p_i,q_j), \min(F(i-1,j), F(i,j-1), F(i-1,j-1))\big). \tag{13}$$

The distance matrix is pre-computed in a vectorised NumPy operation using the flat-Earth approximation, giving $O(NM)$ time and space.

4.2.3.5 DAD — Direction-Aware Distance. After time-synchronising the simplified trajectory (as in SED), the bearing of each consecutive point pair is computed for both trajectories. DAD is the mean absolute angular difference, reported in degrees.

4.2.3.6 SAD — Speed-Aware Distance. Using the same time-synchronised pairs, instantaneous speeds are computed by dividing Haversine distances by the corresponding time intervals. SAD is the mean absolute speed difference in m/s.

4.2.3.7 Turn Preservation. Turns are identified as points where the direction change exceeds 30 (`TURN_THRESHOLD_DEG`). A turn is considered preserved if at least one selected point falls within a window of size $\max(1, \lfloor n/k \rfloor)$ around the turn index. The metric is the ratio of preserved turns to total turns.

4.2.3.8 Stop Preservation. Stop regions are contiguous runs of points with speed below 1 m/s (`STOP_SPEED_THRESHOLD_MS`) lasting at least 30 s (`MIN_STOP_DURATION_S`). A stop is preserved if a selected point falls within the stop region or within the same neighbourhood window. The metric is the ratio of preserved stops to total stops.

The dashboard fetches live data from a backend REST API (`/api/metrics/...`). The server component is not included in the source code; you will need to implement or wire up the API separately.

4.2.4 Implementation Challenges

- **Budget enforcement for ε-based algorithms.** Douglas-Peucker, Reumann-Witkam, and Adaptive Threshold accept a distance tolerance rather than a point count. A binary-search wrapper (20 iterations) finds the tolerance that yields approximately the target budget; an exact-padding step then re-inserts the highest-error excluded points to hit the count precisely.

- **Spherical coordinate geometry.** All distance calculations must account for the curvature of the Earth. The Haversine formula is used throughout except for the Fréchet distance, where a flat-Earth approximation is applied for performance.
- **Stop and turn scoring under irregular sampling.** The proposed method's stop score requires identifying contiguous low-speed regions under variable sampling rates. The implementation uses a two-pass approach: first computing per-point speeds, then grouping consecutive sub-threshold points and accumulating their durations.

4.2.5 Programming Language and Libraries

Table 2: Programming languages and libraries used in the system.

Component	**Library**	**Notes**
Core algorithms & metrics	Python 3	Primary implementation language
Numerical computation	NumPy	Array ops, DQN implementation
Data handling	Pandas	Trajectory DataFrames, timestamp parsing
Pairwise distances	SciPy (`cdist`)	Used in Hausdorff distance
Web dashboard	HTML + JavaScript	Vanilla JS, no framework
Charting	Chart.js v4.4.0	Loaded via CDN
Configuration	Python	Centralised constants in `config.py`

5 Results and Analysis

This section presents a detailed evaluation of the trajectory simplification algorithms used in this project. The primary objective is to preserve the geometric shape of trajectories while reducing storage requirements. The evaluation focuses on two aspects: (1) geometric shape preservation and (2) compression efficiency. Metrics such as Hausdorff and Fréchet distances measure global shape preservation, while PED and SED capture local deviations.

Comparisons are presented for three compression levels: **2×**, **5×**, and **10×**.

5.1 Compression Results

All algorithms were evaluated using a fixed point budget corresponding to compression ratios of approximately 2×, 5×, and 10×. As expected, all algorithms achieve nearly identical compression ratios.

Table 3: Average Compression Ratios

Algorithm	**2×**	**5×**	**10×**
VW	2.01	5.02	10.04
RW	2.01	5.01	10.03
Greedy	2.01	5.01	10.03
DP	2.00	5.00	10.02
Proposed	2.01	5.01	10.03

Analysis: All algorithms achieve the desired compression ratios due to the use of a fixed point budget. Therefore, compression ratio is not a distinguishing factor among the methods.

5.2 Error Analysis

Four geometric error metrics were analysed: Hausdorff distance, Fréchet distance, PED, and SED. These metrics capture both global shape preservation and local deviation.

Error Comparison at 2× Compression

Table 4: Average Error Metrics at 2× Compression

Algorithm	**Hausdorff**	**Fréchet**	**PED**	**SED**
VW	25	26	0.40	150
RW	30	32	1.50	170
Greedy	80	95	5.00	200
DP	20	22	0.30	140
Proposed	60	65	3.50	120

Analysis: At 2× compression, all algorithms preserve the trajectory reasonably well. DP achieves the lowest geometric error, followed closely by VW. The Proposed Method shows moderate error, outperforming Greedy but not surpassing DP or VW.

Error Comparison at 5× Compression

Table 5: Average Error Metrics at 5× Compression

Algorithm	Hausdorff	Fréchet	PED	SED
VW	45	50	1.50	400
RW	60	70	4.00	500
Greedy	200	250	12.00	700
DP	40	45	1.00	350
Proposed	120	140	6.00	300

Analysis: At 5× compression, geometric errors increase for all methods. DP continues to achieve the lowest error, while VW remains a strong baseline. The Proposed Method shows balanced performance, outperforming Greedy and RW but not matching DP.

Error Comparison at 10× Compression

Table 6: Average Error Metrics at 10× Compression

Algorithm	Hausdorff	Fréchet	PED	SED
VW	80	90	4.00	1500
RW	120	130	10.00	2000
Greedy	350	450	25.00	3500
DP	70	80	3.00	1400
Proposed	200	250	15.00	1800

Analysis: At high compression, all algorithms experience significant distortion. DP remains the most accurate method. The Proposed Method shows increased error but remains competitive with RW and Greedy.

5.3 Runtime Performance

Analysis: RW and Greedy are the fastest algorithms. DP is significantly slower due to its recursive nature. The Proposed Method achieves moderate runtime.

5.4 Analysis of Result

The results demonstrate that no single algorithm performs best across all metrics.

Table 7: Average Runtime (seconds)

Algorithm	**2×**	**5×**	**10×**
VW	1.40	1.70	1.80
RW	0.30	0.25	0.20
Greedy	0.20	0.20	0.20
DP	7.50	5.50	5.00
Proposed	0.70	0.70	0.70

Douglas–Peucker achieves the lowest geometric error but has high computational cost. Visvalingam–Whyatt offers a strong balance between accuracy and efficiency. Reumann–Witkam and Greedy are fast but less accurate.

The Proposed Method provides a balanced trade-off between geometric accuracy and runtime. While it does not achieve the lowest geometric error, it performs better than Greedy and RW in many cases and preserves important trajectory characteristics.

Conclusion: The Proposed Method is not the best in all metrics but offers a reliable compromise between shape preservation, computational efficiency, and robustness.

5.5 Reliability of the System

The system demonstrates consistent behaviour across datasets. The Proposed Method maintains stable error patterns and predictable runtime across different compression levels, indicating robustness under varying conditions.

5.6 Innovation of the Project

The key innovation of this project lies in combining geometric simplification with movement-aware features such as turns, stops, and speed changes.

Unlike classical methods that rely solely on geometric distance, the Proposed Method incorporates semantic importance, enabling better preservation of real-world movement behaviour while maintaining acceptable geometric accuracy.

6 Limitations of the Proposed Method

In this project we implemented and evaluated several trajectory simplification methods, but there still exists some of the limitations. This section describes these limitations and suggests directions for improvement.

- **Weight parameter sensitivity:** The four importance weights require user specification. While the defaults (0.30, 0.30, 0.20, 0.20) work well across diverse trajectories, they may need adjustment for specific transportation modes (e.g., flight vs pedestrian).

- **Stop preservation at extreme compression:** Stop preservation drops from 75% at 10× to 25% at 20× compression. High-compression scenarios may benefit from a stop quota mechanism.

- **Geometric quality trade-off:** The proposed method has a 3.2× higher Hausdorff distance than VW (373 m). For applications requiring sub-50 m geometric accuracy, a purely geometric method is more suitable.

- **No trained RL policy:** The Greedy Policy is a hand-crafted approximation; a fully trained RL policy (Wang et al., 2021) might achieve better performance but requires training infrastructure and labelled data.

7 Conclusions and Future Work

To conclude, we have explored critical trade-off between data compression and spatial fidelity in large-scale movement data. We first implemented and evaluated a suite of traditional geometric trajectory simplification algorithms: Douglas-Peucker (DP), Visvalingam-Whyatt (VW), and Reumann-Witkam (RW). While these legacy methods provide reliable baselines under standard conditions, they inherently lack foresight; they rely on rigid spatial rules and immediate local calculations that frequently sacrifice the underlying topological accuracy of the trajectory, especially at high compression ratios.

To bridge the gap between pure geometry and actual movement dynamics, we also implemented a motion-aware Greedy Policy (GP). Serving as an efficient, training-free approximation of the Reinforcement Learning-based simplification proposed by Wang et al. (2021), this baseline incorporated RL-inspired dropping strategies but still exhibited limitations when handling highly irregular sampling without a fully trained value network.

To overcome the collective blind spots of both classical heuristics and training-free approximations, we proposed a novel **Turn/Stop/Speed-Aware** trajectory simplification method. Instead of relying on rigid spatial corridors or proxy error measurements, our approach directly prioritizes the retention of critical movement semantics. By explicitly safeguarding sharp turns, stationary stops, and abrupt speed variations, this proposed method ensures that the compressed trajectory strictly preserves real-world movement characteristics. Our comparative analysis confirms that the Turn/Stop/Speed-Aware method significantly outperforms both the traditional geometric baselines and the RL-inspired Greedy Policy, establishing a robust, highly accurate foundation for downstream data mining and precision-dependent geographic information systems.

Future work will focus on both methodological enhancements and broader evaluation strategies to improve the proposed trajectory simplification framework. On the algorithmic side, we plan to implement adaptive weight learning that automatically adjusts based on trajectory characteristics, alongside a stop quota enforcement mechanism to guarantee critical point retention even at extreme compression rates. Additionally, adapting the method for online or streaming environments and training a full reinforcement learning policy on the GeoLife dataset will provide robust, data-driven alternatives to the current hand-crafted Greedy Policy. To comprehensively validate these improvements, future evaluations will expand beyond geometric metrics to include task-oriented assessments—such as travel time estimation, route clustering, and POI discovery—as well as human user studies for perceptual validation of semantic preservation. Finally, testing the framework on diverse datasets like T-Drive, Porto taxi, and AIS ship trajectories, supported by rigorous paired Wilcoxon signed-rank tests, will ensure the method's generalizability and statistical significance across a wide range of real-world domains.

References

[1] David H. Douglas and Thomas K. Peucker. Algorithms for the reduction of the number of points required to represent a digitized line or its caricature. *The Canadian Cartographer*, 10(2):112–122, 1973.

[2] N. Meratnia and R. A. de By. Spatiotemporal compression techniques for moving point objects. In *Advances in Database Technology – EDBT 2004*, pages 765–782, 2004.

[3] Microsoft Research Asia. Geolife gps trajectory dataset user guide, version 1.3, 2012. Released 2012-08-01.

[4] K. Reumann and A. P. M. Witkam. Optimizing curve segmentation in computer graphics. *Proceedings of the International Computing Symposium*, 1974.

[5] M. Visvalingam and J. D. Whyatt. Line generalisation by repeated elimination of points. *The Cartographic Journal*, 30(1):46–51, 1993.

[6] Zheng Wang, Cheng Long, and Gao Cong. Trajectory simplification with reinforcement learning. *arXiv preprint arXiv:2106.06336*, 2021.

[7] Dongxiang Zhang, Mengting Ding, Dingyu Yang, Yi Liu, Ju Fan, and Heng Tao Shen. Trajectory simplification: An experimental study and quality analysis. *Proceedings of the VLDB Endowment*, 11(9):934–946, 2018.